

1 Постановка задачі та вимоги до програми

1.1 Стан питання та обґрунтування теми

Об'єктом дослідження є процес автоматичного групування природномовних текстів. Предмет дослідження становлять методи попередньої обробки україномовних документів, їх векторного подання та кластеризації, а також співвідношення якості групування й обчислювальних витрат. Практичне завдання полягає у створенні локального інструмента, який дозволяє проводити порівнянні експерименти за єдиною схемою та зберігати результати для подальшого аналізу.

У текстових колекціях одна тема може бути виражена різними словами, а одне слово — мати різні значення. Для української мови додаткову складність створює словозміна: форми одного слова без нормалізації збільшують словник ознак. Тому якість кластеризації залежить не лише від алгоритму групування, а й від очищення тексту, лематизації та способу формування векторів. Вибір цих етапів має перевірятися експериментально.

TF-IDF є інтерпретованою базовою моделлю, проте спільна тематика не завжди супроводжується збігом словника. Word2Vec навчає вектори слів, а FastText використовує інформацію про частини слова [3; 4]. Doc2Vec формує подання документів [5]. Sentence-BERT пропонує подання речень для семантичного порівняння [6]. Ці підходи розв'язують різні аспекти задачі, тому немає підстав наперед оголошувати один із них найкращим для будь-якого українського корпусу.

Готові засоби вже підтримують значну частину необхідних операцій. scikit-learn надає алгоритми кластеризації та оцінювання; Gensim — моделі векторизації; Orange — візуальне компонування обробки тексту [2–4; 7]. Отже, внесок роботи полягає не у створенні кластеризації як нового класу методів, а в інтеграції компонентів у спеціалізований інструмент і перевірці власної організації паралельних перезапусків K-Means.

1.2 Невирішене питання та мета роботи

Для конкретного корпусу залишається відкритим питання, яка комбінація обробки, векторизації та кластеризації забезпечує прийнятне тематичне групування за доступних ресурсів. Мета роботи — розробити й перевірити інструментальні засоби для такого порівняння. Завдання охоплюють імпорт даних, уніфікацію конвеєра, реалізацію Parallel K-Means, розрахунок метрик, вимірювання ресурсів і формування звітів. Перевага вдосконаленого підходу розглядається як гіпотеза, що потребує перевірки.

1.3 Критичне зіставлення підходів

Відмінні оцінки методів не обов'язково суперечать одна одній: високий Silhouette характеризує геометрію векторного простору, тоді як F1 або V-measure — узгодженість з еталонною розміткою. Так само менша inertia у K-Means не гарантує кращого відтворення тем. У цій роботі ці критерії розглядаються спільно, а результати різних корпусів і налаштувань не ототожнюються.

Засіб	Корисні можливості	Що необхідно для цієї роботи
scikit-learn	Кластеризація та метрики	Об'єднати з українською обробкою й обліком ресурсів
Gensim	Word2Vec, FastText, Doc2Vec	Додати інтерфейс експерименту, оцінювання та звіти
Orange Text Mining	Візуальний аналіз корпусів	Перевірити власну схему Parallel K-Means у керованому сценарії

1.4 Функціональні та нефункціональні вимоги

За технічним завданням [1] програма повинна завантажувати TXT, CSV і JSON, очищати та лематизувати тексти, підтримувати налаштовувані стоп-слова, реалізовувати TF-IDF, Word2Vec, FastText і Doc2Vec. Для кластеризації потрібні K-Means, агломеративний метод і DBSCAN із доступними користувачу параметрами. Результат експерименту має включати оцінки якості, часові та ресурсні характеристики, візуалізації й експорт у CSV, JSON та PDF або HTML.

До розширених вимог належать вибір CBOW або Skip-gram, зважене TF-IDF усереднення векторів слів, метод ліктя, аналіз k-distance, імпорт навчених моделей і збереження міток кожного документа. Їх наявність оцінюється окремо від базового конвеєра. Критерієм приймання є не лише формування числової таблиці, а й правильність обробки некоректних даних, зрозумілі повідомлення та можливість повторного аналізу збережених результатів.

ТЗ визначає мінімум 8 ГБ оперативної пам'яті, 1 ГБ вільного диска та процесор від 2 ГГц; передбачає Windows, Linux і macOS. Це цільові вимоги, а не підтверджені результати випробувань усіх платформ. Зазначене в ТЗ обмеження Python 3.8+ потребує уточнення: код використовує сучасні анотації типів, а доступність сумісних версій залежностей слід перевіряти для конкретного середовища. Практичний запуск перевірено в середовищі, наведеному в розділі 4.

Висновок аналізу: доцільно використати бібліотечні алгоритми, зосередивши власну розробку на узгодженні етапів, українській попередній обробці, керуванні експериментами та паралельних незалежних ініціалізаціях. Вимоги до надійності включають незмінність початкового корпусу, перевірку формату та збереження кожного завершеного запуску в окремій папці.

2 Проектування програми

2.1 Загальний підхід до проектування

Програму спроектовано як настільний застосунок Python із модульним поділом відповідальності. Інтерфейс збирає параметри й показує результати; сервіс керує експериментом; спеціалізовані модулі реалізують обробку даних, векторизацію, кластеризацію та оцінювання. Такий поділ дозволяє перевіряти обчислювальний сценарій без запуску графічного вікна. Основний потік перетворення даних наведено на рис.1.

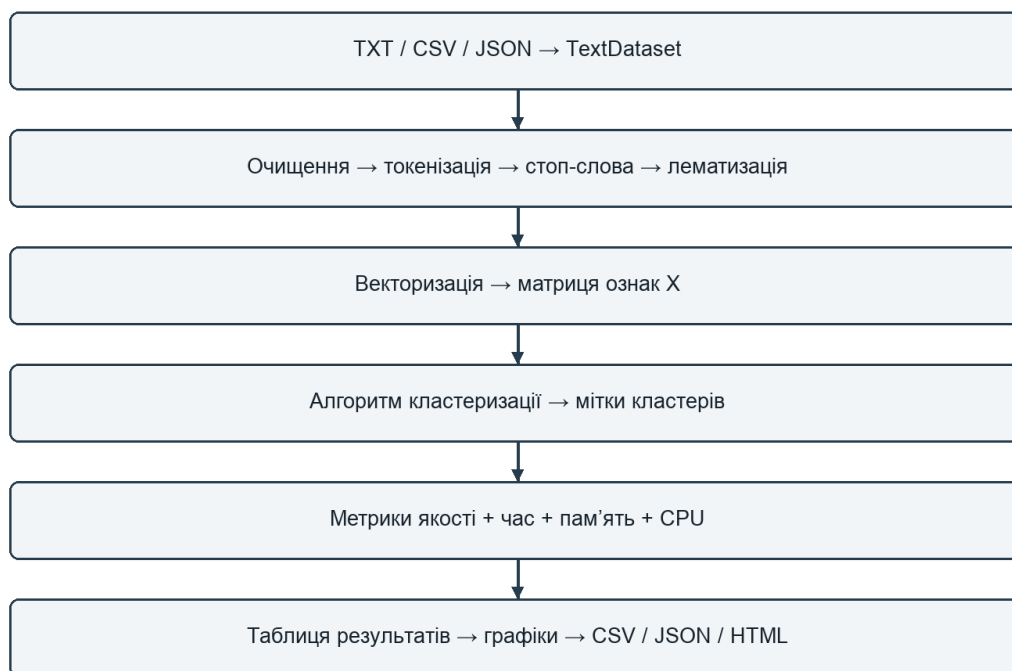


Рисунок 1 – Конвеєр проведення експерименту

Порядок етапів є суттєвим: спочатку формується спільний список документів і токенів, потім для кожного обраного векторизатора обчислюється матриця ознак. На цій матриці послідовно виконуються вибрані алгоритми. Це виключає випадкову зміну корпусу між алгоритмами в межах одного запуску. Еталонні мітки використовуються під час оцінювання, а не для навчання кластеризаторів.

У проектуванні застосовано принцип єдиної відповідальності та адаптери над бібліотечними моделями. Уніфіковані операції `fit_transform` і `fit_predict` зменшують залежність керівного коду від конкретного методу. Збереження налаштувань підтримує простежуваність експерименту. Повну відповідність окремому стандарту життєвого циклу ПЗ не заявлено: для неї потрібні додаткові регламенти, протоколи та формальні процедури приймання.

2.2 Зовнішнє проектування та сценарії користувача

Основним користувачем є дослідник, який володіє базовими поняттями обробки тексту та машинного навчання. Типовий сценарій починається з вибору файлу та назв колонок. Далі користувач обирає один або кілька векторизаторів і кластеризаторів, задає параметри й запускає експеримент. Після завершення він порівнює метрики, переглядає графіки та відкриває збережені звіти. Приклад налаштування головного вікна слід подати так, як наведено на рис.2.

МІСЦЕ	ДЛЯ	СКРІНШОТА
Вставити скріншот головного вікна Text Clustering Research. Обрати datasets/demo_texts.csv; колонки text і label; TF-IDF; K-Means, Parallel K-Means, Hierarchical, DBSCAN. Показати K = 5, runs = 3, eps = 0.5, min_samples = 5, 8 перезапусків і 4 потоки. Увімкнути стоп-слова, графіки та рекомендацію K; лематизацію вимкнути. Розгорнути блоки налаштувань; приховати зайві частини робочого столу.		

Рисунок 2 – Головне вікно та налаштування експерименту

Параметри кількості кластерів використовуються лише алгоритмами, які потребують K. Для DBSCAN суттєвими є eps та min_samples, а для агломеративного методу — linkage. Налаштування k-means++ або випадкової ініціалізації впливає на початкові центри. Поле максимальної кількості текстів дає змогу провести попередній запуск на перших N документах, проте такий відбір не є випадковою репрезентативною вибіркою.

Інтерфейс побудовано засобами tkinter і ttk. У ньому передбачені згортані секції, таблиця результатів, текстовий висновок, вікно порівняння та перегляд попередніх експериментів. Перегляд збереженого запуску має окрему цінність: користувач отримує доступ до вже обчислених показників без повторного навчання моделей.

Зовнішніми результатами роботи є локальні файли та елементи інтерфейсу. Вебсервер, реєстрація користувачів і мережеве сховище в цій архітектурі не потрібні. Мережевий доступ може знадобитися під час першого завантаження моделі Sentence-BERT, якщо її ще немає в кеші.

2.3 Керування виконанням і обробка помилок

Тривалі обчислення виконуються у фоновому потоці. Метод `_run` формує `ExperimentOptions`, запускає `_worker` і планує періодичне опитування черги подій. Сервіс надсилає повідомлення про перебіг роботи через `callback`; події `progress`, `done` та `error` потрапляють у `queue.Queue`. Метод `_poll`, викликаний через `after` з інтервалом 120 мс, оновлює віджети в головному потоці. Взаємодію наведено на рис.3.

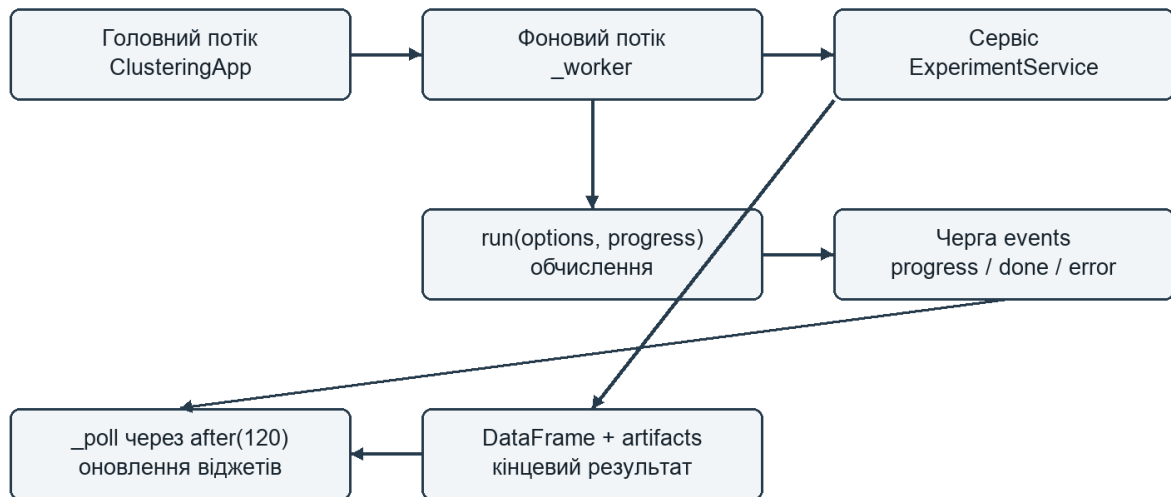


Рисунок 3 – Взаємодія графічного інтерфейсу та фонового обчислення

Черга відокремлює виконання обчислень від операцій з віджетами. Завдяки цьому сервіс не потребує посилань на вікна або таблиці `tkinter`. Поверненими значеннями `run` є `DataFrame` із результатами та словник `artifacts` із шляхами до створених файлів і підсумковим текстом. Інтерфейс використовує ці дані для заповнення таблиці та показу графіків.

На рівні завантаження перевіряються існування файлу, розширення, наявність текстової колонки й допустимість обмеження розміру корпусу. Сервіс вимагає щонайменше два документи та непорожній набір токенів після очищення. Помилка окремого кластеризатора записується в поле `error`, після чого виконання наступних комбінацій може тривати.

Межа ізоляції помилок не охоплює весь конвеєр: створення векторизатора, векторизація та рекомендація `K` виконуються поза внутрішнім блоком обробки помилки алгоритму. Відсутня залежність або збій на цих етапах може припинити весь поточний запуск. Крім того, профілювання потребує гарантованого звільнення ресурсів через `try/finally`. Ці обмеження враховуються під час оцінювання надійності.

2.4 Проектування вдосконаленого підходу

Власний `ParallelKMeansClusterer` організовує паралельний запуск кількох незалежних екземплярів бібліотечного K-Means. Для R перезапусків задаються R різних початкових зерен генератора випадкових чисел. Кожен кандидат використовує $n_init = 1$ і повертає пару з $inertia$ та міток. Остаточним результатом є мітки кандидата з найменшою внутрішньокластерною сумою квадратів. Схему наведено на рис.4.

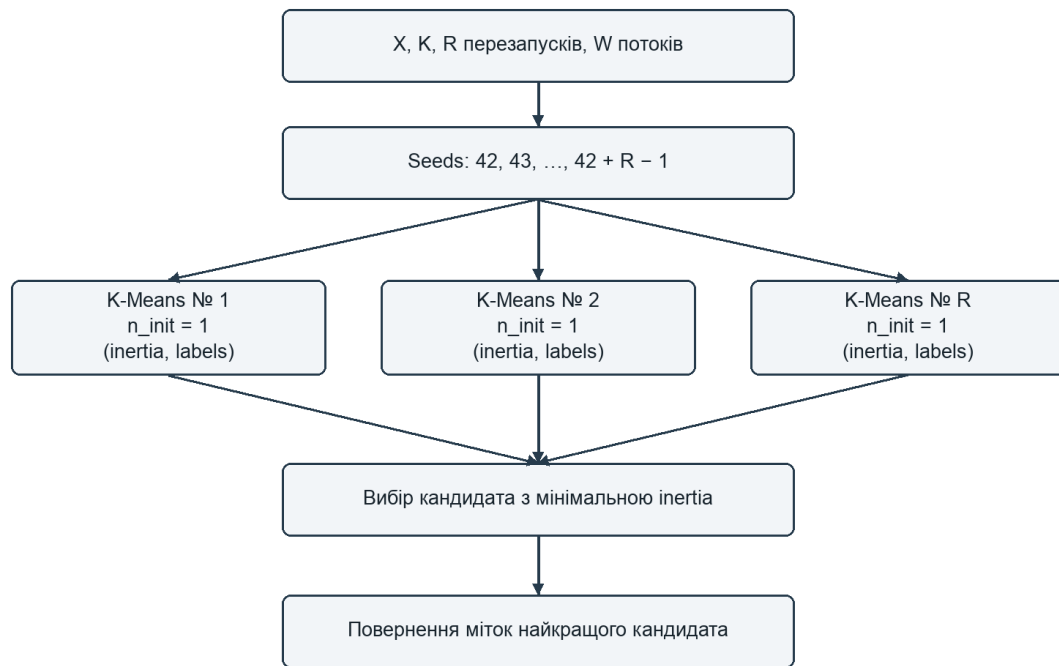


Рисунок 4 – Паралельний багатостартовий K-Means

`ThreadPoolExecutor` створює не більше $\min(W, R)$ робочих потоків. За замовчуванням $R = 8$, $W = 4$, а зерна утворюють послідовність від 42 до 49. Перед виконанням перевіряються $K \geq 2$, $K \leq N$ та додатність R і W . Спільна матриця X передається кандидатам для читання; кожен кандидат створює власну модель.

Це вдосконалення організації пошуку початкового розв'язку, а не нова цільова функція K-Means. Його результат не гірший за $inertia$ серед виконаних кандидатів, проте не обов'язково кращий за зовнішніми метриками. Прискорення потрібно зіставляти з послідовним виконанням тих самих R стартів: порівняння з одним стартом одночасно змінює і обсяг роботи, і спосіб її виконання. Для малого корпусу накладні витрати потоків можуть переважати виграш.

3 Компонентна структура програми

3.1 Розробка інструментальних засобів для дослідження кластеризації текстів

Компонентна структура відображає фактичні модулі проекту та основний графічний сценарій. Точка входу `main.py` за замовчуванням запускає інтерфейс, а параметр `--batch` активує окремий пакетний сценарій. Центральним координатором інтерактивного режиму є `ExperimentService`. Логічні компоненти та напрямки обробки наведено на рис.5; стрілки позначають основні потоки даних, а не повний граф імпортів Python.

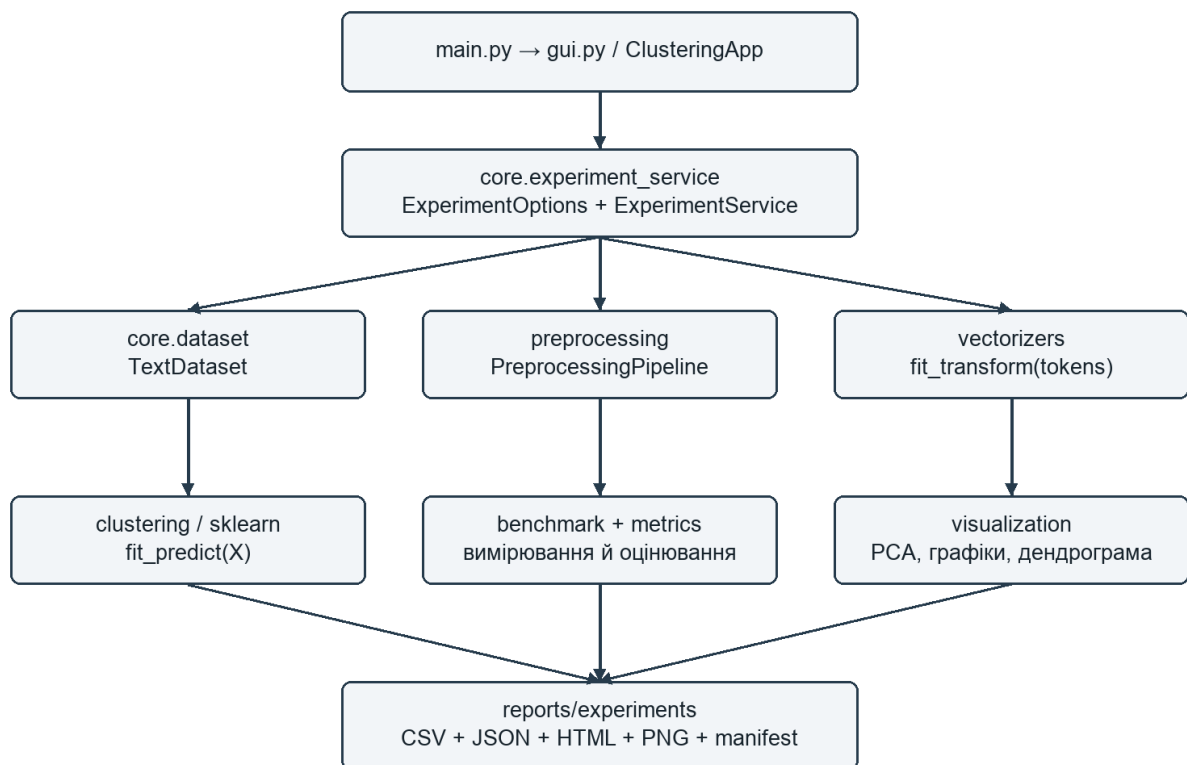


Рисунок 5 – Компонентна структура інтерактивного режиму

Пакетний сценарій використовує `ExperimentRunner`, `Config` та `ExperimentResult`. Він не тотожний графічному: має інший спосіб побудови моделей, імпортує залежності заздалегідь і формує окремий звіт. Тому перелік можливостей інтерфейсу не слід автоматично переносити на `--batch`. У межах контрольних випробувань основним є сервісний шлях, який використовує графічний застосунок.

Залежності між етапами передаються через прості структури: список текстів, списки токенів, матрицю NumPy, масив міток, словники показників і DataFrame. Спільний контракт зменшує зв'язність між модулями. Водночас вибір конкретних реалізацій поки задається умовними фабричними методами сервісу; підключення нового алгоритму потребує доповнення відповідної гілки та списку доступних методів.

3.2 Призначення основних компонентів

Компонент	Відповідальність та основний контракт
gui.py	ClusteringApp: налаштування, фоновий запуск, черга подій, таблиця, перегляд графіків та історії.
core/dataset.py	TextDataset.load: читання TXT/CSV/JSON; тексти та необов'язкові еталонні мітки.
preprocessing	TextCleaner, SimpleTokenizer, StopwordRemover, Lemmatizer; process(text) → список токенів.
vectorizers	Адаптери TF-IDF, Word2Vec, FastText, Doc2Vec, Sentence-BERT; fit_transform → матриця ознак.
clustering	Адаптери алгоритмів і ParallelKMeansClusterer; fit_predict → мітки кластерів.
benchmark	Timer, MemoryTracker, CPUMonitor, BenchmarkRunner; результат операції та агреговані вимірювання.
metrics	evaluate_internal та evaluate_external: внутрішні та зовнішні оцінки кластеризації.
visualization	Побудова проєкцій і графіків; у GUI використовується PCA.
core/experiment_service.py	Послідовність експерименту, вибір моделей, рекомендація K, експорт та повторне читання.

3.3 Реалізація обробки та векторизації

TextCleaner переводить рядок у нижній регістр, видаляє URL, цифри й символи, що не входять до дозволеного набору літер і пробілів. SimpleTokenizer розділяє рядок за пробілами. Далі за потреби застосовуються базовий і користувацький списки стоп-слів та український морфологічний аналізатор rymorphy3. Видалення стоп-слів передуює лематизації; це слід враховувати при додаванні словоформ до власного списку.

TF-IDF обмежує словник 5000 ознаками та перетворює розріджену матрицю на щільний масив. Word2Vec і FastText усереднюють вектори слів, а для порожнього набору повертають нульовий вектор. Doc2Vec використовує унікальні теги документів. Ці моделі навчаються на поточному корпусі; для 50 коротких текстів обсяг даних недостатній для загальних висновків про їхню семантичну якість.

Sentence-BERT завантажує paraphrase-multilingual-MiniLM-L12-v2 і кодує рядки, зібрані з оброблених токенів. Отже, модель отримує вже очищений і, за вибором користувача, лематизований текст. Вплив такого перетворення на контекстні подання потребує окремого порівняння з кодуванням початкових речень.

3.4 Організація даних та збереження результатів

Вхідний TXT містить один документ у непорожньому рядку. У CSV і JSON користувач задає назву поля тексту та, за наявності, поля розмітки. Обмеження `max_documents` залишає перші `N` записів. Поточний завантажувач перетворює значення текстової колонки на рядки; пропуски в табличних даних потребують окремої перевірки, щоб службові значення не ставали текстами. Основні структури наведено на рис.6.

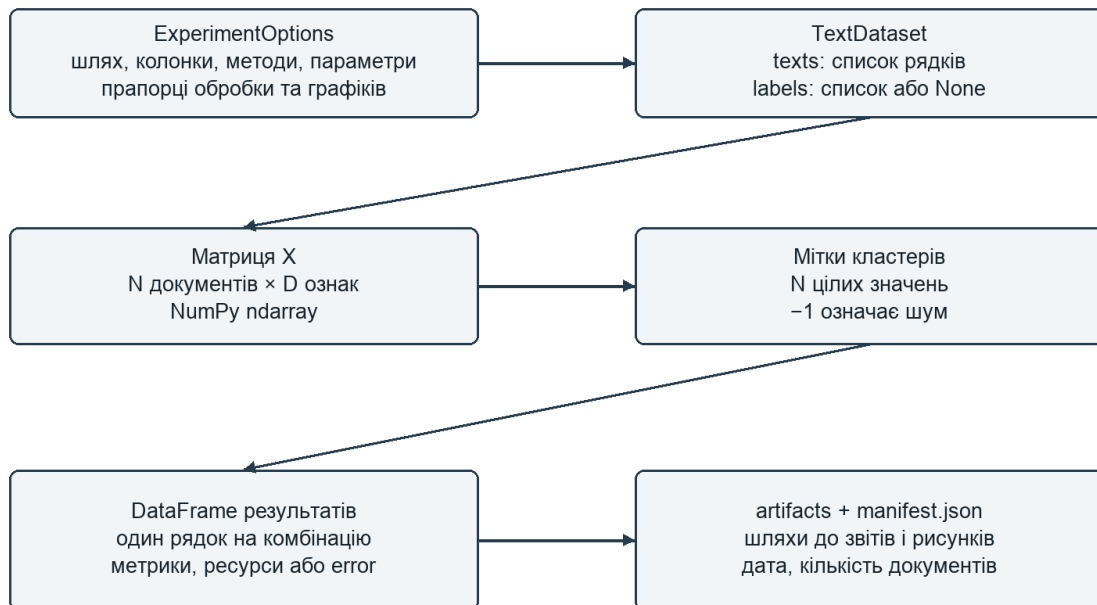


Рисунок 6 – Структури даних та результати експерименту

Кожен рядок результатів містить назву корпусу, векторизатор, алгоритм, кількість документів, знайдених кластерів і шумових точок, час та пам'ять векторизації й кластеризації, CPU та метрики якості. Рекомендоване `K` визначається окремим перебором `K`-Means від 2 до $\min(10, N - 1)$ за Silhouette; воно є рекомендацією і не замінює автоматично введене користувачем `K`.

Файли групуються в `reports/experiments/experiment_YYYYMMDD_HHMMSS`. CSV зручний для табличного аналізу, JSON — для повторного читання програмою, HTML — для перегляду звіту. Конфігурація серіалізується з `ExperimentOptions` у плоский JSON, а `manifest.json` пов'язує результати з рисунками. Це відрізняється від вкладеного прикладу конфігурації у ТЗ. Поточний експорт містить агреговані показники, але не таблицю відповідності кожного тексту його кластеру.

Окрема папка запуску зменшує ризик випадкового перезапису, однак не замінює резервного копіювання. Для довготривалої відтворюваності доцільно додати копію або хеш корпусу, версії залежностей, параметри випадковості, відносні шляхи та унікальний ідентифікатор запуску замість часової позначки з точністю до секунди.

4 Тестування програми

4.1 Методика та середовище перевірки

Перевірку виконано 20 вересня 2026 року. Середовище контрольного запуску: macOS-27.0-x86_64-i386-64bit, Python 3.12.13. Використано наявне віртуальне середовище проекту та демонстраційний корпус datasets/demo_texts.csv із 50 україномовних документів. Додатково перевірено імпорт еквівалентних TXT і JSON. Це функціональна перевірка на малому корпусі, а не навантажувальна атестація заявлених платформ.

У репозиторії є два автоматичні тести попередньої обробки: видалення URL і переведення літер у нижній регістр. Команда `python -m pytest tests/ -q` завершилася результатом 2 passed. Окремий контрольний сценарій виконав сервісний конвеєр і перевіряв збереження та повторне читання. Виконані перевірки наведено в таблиці 1.

Таблиця 1 – Фактично виконані перевірки

Перевірка	Очікуваний результат	Фактичний результат
Очищення URL	URL вилучено, слово «сьогодні» збережено	Пройдено
Нижній регістр	«ВЕЛИКІ літери» → «великі», «літери»	Пройдено
Імпорт TXT	50 документів	Пройдено
Імпорт CSV	50 документів	Пройдено
Імпорт JSON	50 документів	Пройдено
Повний цикл без лематизації	4 комбінації, 3 повтори	Пройдено
Відновлення	4 рядки без перерахунку	Пройдено
Відсутній файл	FileNotFoundError	Пройдено
Відсутня колонка	ValueError	Пройдено

Для інтеграційного запуску обрано TF-IDF і чотири алгоритми: K-Means, Parallel K-Means, агломеративний метод та DBSCAN. Параметри: $K = 5$, $\text{linkage} = \text{ward}$, $\text{eps} = 0.5$, $\text{min_samples} = 5$, $R = 8$, $W = 4$. Увімкнено стоп-слова, графіки та рекомендацію K; лематизацію вимкнено. Кожна вимірювана операція виконувалася тричі; часові значення в таблиці є середніми цих повторів.

Контрольний сценарій перевіряв чотири рядки результатів без поля `error`, створення комплекту артефактів і повторне завантаження чотирьох записів. Два негативні сценарії підтвердили `FileNotFoundError` для відсутнього файлу та `ValueError` для відсутньої текстової колонки. Візуальна взаємодія користувача з GUI цими перевітками не покривається.

4.2 Результати контрольного експерименту

Перший запуск із лематизацією завершився ValueError: для pymorphy3 відсутній словник uk, доступний лише ru. Залежність pymorphy3-dicts-uk зазначена у requirements.txt, проте в перевіреному середовищі її немає. Наступний контрольний запуск виконано без лематизації. Отже, її працездатність у цьому середовищі не підтверджено; перед повним прийманням потрібно встановити словник і повторити перевірку.

Таблиця 2 – Результати TF-IDF на демонстраційному корпусі

Алгоритм	Кластери / шум	Silhouette	F1 macro	Час, с
kmeans	5 / 0	0.0026	0.2390	0.0040
parallel_kmeans	5 / 0	0.0069	0.2364	0.0340
hierarchical	5 / 0	0.0106	0.3029	0.1392
dbscan	0 / 50	-1.0000	0.0000	0.0028

Графік середнього часу кластеризації, сформований під час цього запуску, наведено на рис.7. Час векторизації вимірюється окремо; підготовка корпусу, рекомендація K, завантаження моделі та побудова рисунків не входять до показника clustering_seconds. Тому цей показник не дорівнює тривалості всього експерименту.

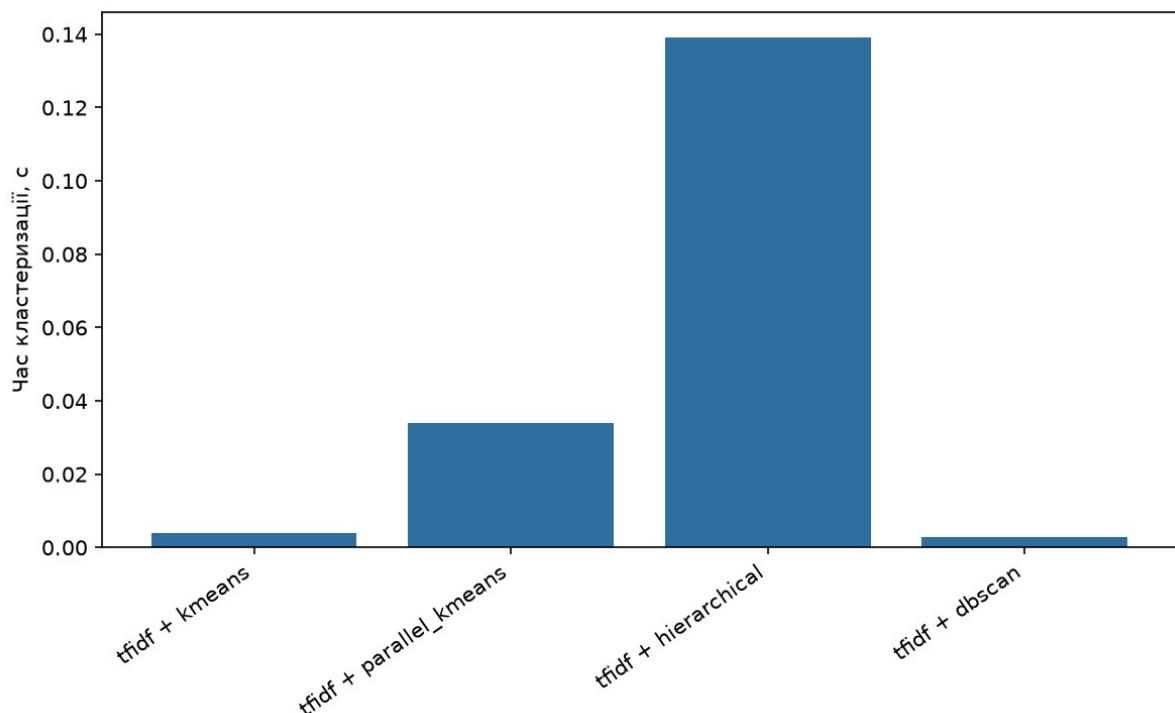


Рисунок 7 – Порівняння часу кластеризації у контрольному запуску

Найбільший Silhouette у цьому запуску отримано для hierarchical (0.0106), а найменший час — для dbscan (0.0028 с). Ці спостереження стосуються лише вибраного корпусу й параметрів. Три повтори з фіксованими початковими умовами оцінюють коливання часу, але не є трьома незалежними дослідженнями стійкості якості.

Якщо DBSCAN відносить усі документи до шуму, це означає непридатність поточних параметрів цільності для сформованого простору, а не програмний збій. Значення -1 у внутрішніх метриках є службовим результатом реалізації для неможливого оцінювання. Особливо для Davies–Bouldin його не можна трактувати як найкращу оцінку.

4.3 Візуальна та метрична перевірка

Дендрограму контрольного запуску з `linkage = ward` наведено на рис.8. Вона показує послідовність об'єднань і відстані між групами, а не точність тематичної класифікації. За 50 документів програма приховує підписи листків. На відміну від таблиці метрик, рисунок дає змогу візуально оцінити структуру об'єднань, проте не замінює кількісної перевірки.

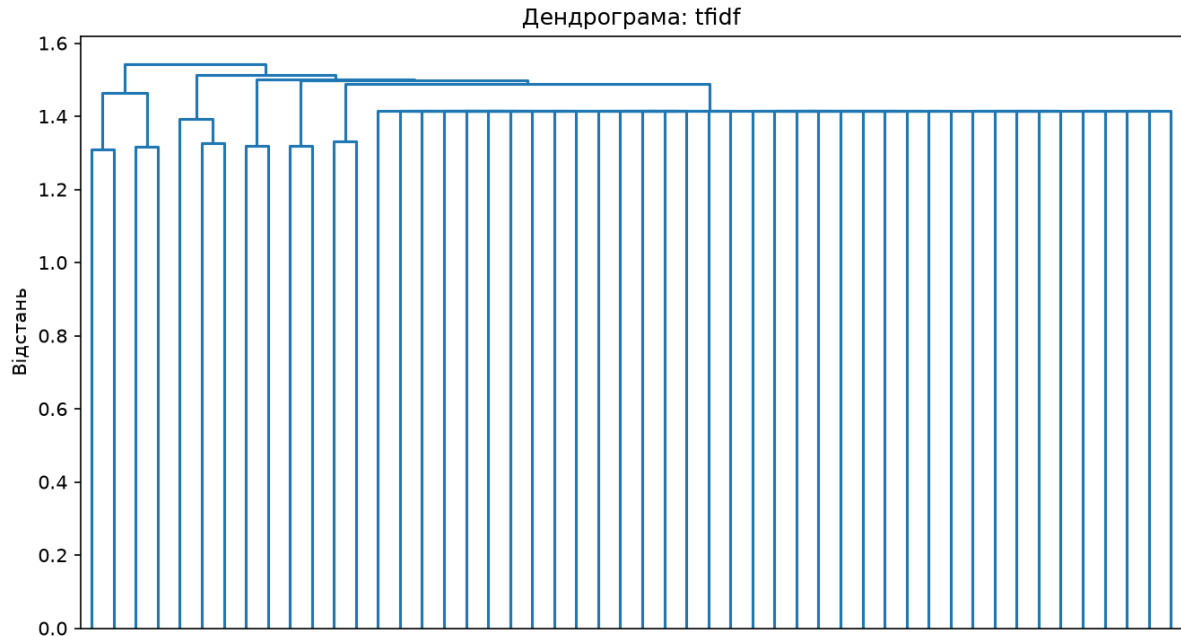


Рисунок 8 – Дендрограма TF-IDF для агломеративного аналізу

У поточній реалізації дендрограма завжди використовує `ward`, навіть коли користувач обирає інший `linkage` для кластеризатора. Для цього контрольного запуску суперечності немає, оскільки обрано `ward`. Для `single`, `complete` та `average` необхідно синхронізувати параметри рисунка й моделі або явно зазначати, що рисунок є окремим аналізом.

F1 розраховується після взаємно однозначного зіставлення кластерів із класами за максимумом збігів через `linear_sum_assignment`. ARI, NMI та V-measure використовують початкові номери кластерів. Перед зовнішнім оцінюванням шум вилучається; отже, разом із метриками обов'язково аналізується `noise_documents`. У внутрішніх оцінках за наявності кількох кластерів шум окремо не фільтрується, і мітка `-1` може брати участь як група.

Пам'ять обліковується через `tracemalloc` як пік відстежуваних виділень, а не як повне споживання RAM процесом. CPU процесу нормується на кількість логічних ядер, тоді як показники по ядрах є системними вибірками й можуть містити активність інших програм. Інтервал моніторингу 0.1 с обмежує точність для коротких операцій. Ці особливості забороняють трактувати показники як точний апаратний профіль кожного алгоритму.

4.4 Перевірка інтерфейсу та відповідності технічному завданню

Для завершення візуальної перевірки слід відкрити збережений контрольний експеримент через «Попередні експерименти...», перевірити таблицю та вікно порівняння. Результат цього ручного кроку необхідно показати, як наведено на рис.9. Місце для знімка залишено навмисно; наявність описаного сценарію не означає, що ручну перевірку вже виконано.

МІСЦЕ	ДЛЯ	СКРІНШОТА
Вставити скріншот після відкриття збереженого контрольного запуску з practice_report/qa/runs/experiments. Показати чотири рядки TF-IDF, стовпці алгоритму, Silhouette, часу й пам'яті та загальний висновок. За потреби скористатися кнопкою «Порівняти результати». Числа мають відповідати таблиці 2; скріншот нового запуску матиме інші часові показники.		

Рисунок 9 – Відображення результатів та порівняння методів

Таблиця 3 – Стан виконання ключових вимог за аналізом коду

Вимога	Стан поточної версії
TXT/CSV/JSON; базова обробка	Реалізовано; базовий шлях перевірено
TF-IDF; K-Means; ієрархія; DBSCAN	Реалізовано; виконано контрольний запуск
Word2Vec, FastText, Doc2Vec, SBERT	Є реалізації; повний цикл у цій перевірці не запускався
CBOW/Skip-gram; зважене усереднення	Немає користувацького вибору; реалізовано просте усереднення
Автоматичний підбір параметрів	Є рекомендація K за Silhouette; немає ліктя і k-distance
Час навчання й передбачення окремо	Вимірюється fit_predict загалом
Експорт і збереження моделей	CSV/JSON/HTML є; довільний імпорт моделей і експорт міток документів відсутні
Резервування та кросплатформність	Окремі папки є; резервні копії й усі ОС не перевірені

Наступні випробування мають охопити порожні та однакові тексти, $K \geq N$, частково відсутні значення, недоступну модель, помилку запису, великі корпуси й повторюваність випадкових моделей. Для порівняння швидкодії Parallel K-Means потрібен послідовний контроль із тими самими вісьмома зернами та обмеженням внутрішніх потоків чисельних бібліотек.

5 Висновки

У межах практики розглянуто й перевірено програмний інструмент для дослідження кластеризації природномовних текстів. Аналіз предметної області показав, що оцінювати потрібно весь конвеєр: спосіб попередньої обробки та векторизації визначає простір, у якому працює кластеризатор. Вибір методу лише за одним показником або за результатом одного малого корпусу не дає достатнього обґрунтування для практичного застосування.

Спроекована модульна структура відокремлює графічний інтерфейс, керування експериментом, завантаження даних, мовну обробку, алгоритми й оцінювання. Зовнішні аспекти проектування охоплюють сценарії налаштування, запуску, порівняння та повторного перегляду. Внутрішні аспекти охоплюють контракти компонентів, передачу матриці ознак і міток, фонове виконання, чергу подій та організацію файлових результатів. Згенеровані схеми відображають ці рішення та пов'язують структуру з фактичним кодом.

Власний Parallel K-Means реалізує конкурентне виконання незалежних стартів із вибором мінімальної inertia. Практична цінність цього рішення полягає у керованому багатостартовому пошуку й можливості експериментально оцінити вплив кількості перезапусків та потоків. Його перевага за швидкодією або тематичною якістю не є автоматичною: вона залежить від обсягу даних, параметрів і середовища виконання.

Виявлено відсутність українського словника rymorphy3; контрольний конвеєр перевірено з вимкненою лематизацією. Під час перевірки успішно виконано два наявні модульні тести, імпорт трьох форматів, сервісний запуск чотирьох комбінацій TF-IDF з алгоритмами кластеризації, повторне читання результатів і два негативні сценарії завантаження. Програма сформувала метрики, графіки, конфігурацію та звіти. Це підтверджує працездатність перевіреного базового сценарію, але не є доказом повної відповідності всім вимогам ТЗ або придатності до великих промислових корпусів.

Подальше вдосконалення має включати вибір архітектури Word2Vec, зважену агрегацію, метод ліктя й k-distance, експорт міток документів, узгодження дендрограми з linkage та надійне завершення профілювання після винятків. Для відтворюваності потрібні фіксація версій, хеш корпусу та керування всіма джерелами випадковості. Окремим дослідницьким етапом має стати оцінювання на кількох розмічених україномовних корпусах різних обсягів і тематик.

Таким чином, результат практики становить основу для першого й третього розділів пояснювальної записки: обґрунтовано постановку задачі та місце роботи серед наявних підходів, описано процес проектування й компонентну реалізацію, наведено підтверджені результати перевірки та визначено межі поточної версії.

Список використаних джерел

1. Дослідження методів кластеризації природомовних текстів. Технічне завдання. Локальний документ проекту «Технічне завдання.docx».
2. scikit-learn developers. Clustering. Офіційна документація. <https://scikit-learn.org/stable/modules/clustering.html>
3. Gensim. models.word2vec — Word2vec embeddings. Офіційна документація. <https://radimrehurek.com/gensim/models/word2vec.html>
4. Bojanowski P., Grave E., Joulin A., Mikolov T. Enriching Word Vectors with Subword Information. Transactions of the Association for Computational Linguistics. 2017. Vol. 5. P. 135–146. <https://aclanthology.org/Q17-1010/>
5. Le Q., Mikolov T. Distributed Representations of Sentences and Documents. Proceedings of the 31st International Conference on Machine Learning. 2014. Vol. 32. P. 1188–1196. <https://proceedings.mlr.press/v32/le14.html>
6. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP. 2019. P. 3982–3992. DOI: 10.18653/v1/D19-1410. <https://aclanthology.org/D19-1410/>
7. Orange Data Mining. Widget Catalog; Text Mining. Офіційний каталог компонентів. <https://orangedatamining.com/widget-catalog/>
8. scikit-learn developers. Selecting the number of clusters with silhouette analysis on KMeans clustering. https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis
9. Text Clustering Research. Вихідний код проекту: README.md, gui.py, core, preprocessing, vectorizers, clustering, metrics, benchmark, visualization, tests. Версія робочої папки, перевірена 20.09.2026.

Дата звернення до електронних джерел: 20.09.2026.